

AI ASSISTED CODING

ASSIGNMENT-13

NAME: Varshitha. G

HTNO: 2303A51103

Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

Sample Legacy Code:

Legacy script with repeated logic

```
print("Area of Rectangle:", 5 * 10)
```

```
print("Perimeter of Rectangle:", 2 * (5 + 10))
```

```
print("Area of Rectangle:", 7 * 12)
```

```
print("Perimeter of Rectangle:", 2 * (7 + 12))
```

```
print("Area of Rectangle:", 10 * 15)
```

```
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

```
# refactor the code with reusable function and no duplication and add docstring to all functions
def area_of_rectangle(length: float, breadth: float) -> float:
    """This function calculates the area of a rectangle given its length and breadth.
    Parameters:
    length (float): The length of the rectangle
    breadth (float): The breadth of the rectangle
    Returns:
    float: The area of the rectangle"""
    return length * breadth

def perimeter_of_rectangle(length: float, breadth: float) -> float:
    """This function calculates the perimeter of a rectangle given its length and breadth.
    Parameters:
    length (float): The length of the rectangle
    breadth (float): The breadth of the rectangle
    Returns:
    float: The perimeter of the rectangle"""
    return 2 * (length + breadth)

# Example usage:
rectangles = [(5, 10), (7, 12), (10, 15)]
for length, breadth in rectangles:
    area = area_of_rectangle(length, breadth)
    perimeter = perimeter_of_rectangle(length, breadth)
    print(f"Area of Rectangle with length {length} and breadth {breadth}: {area}")
    print(f"Perimeter of Rectangle with length {length} and breadth {breadth}: {perimeter}")
```

BLEMS	OUTPUT	TERMINAL	PORTS
-------	--------	----------	-------

TERMINAL

```
Area of Rectangle with length 5 and breadth 10: 50
Perimeter of Rectangle with length 5 and breadth 10: 30
Area of Rectangle with length 7 and breadth 12: 84
Perimeter of Rectangle with length 7 and breadth 12: 38
Area of Rectangle with length 10 and breadth 15: 150
Perimeter of Rectangle with length 10 and breadth 15: 50
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

- Sample Legacy Code:

Legacy script with inline repeated logic

```
price = 250
```

```
tax = price * 0.18
```

```
total = price + tax
```

```
print("Total Price:", total)
```

```
price = 500
```

```
tax = price * 0.18
```

```
total = price + tax
```

```
print("Total Price:", total)
```

```
44 # refactor the code with a function calculate_total(price) that can be reused for mu
45 def calculate_total(price: float) -> float:
46     """This function calculates the total price including tax for a given price.
47     Parameters:
48     price (float): The original price of the item
49     Returns:
50     float: The total price including tax"""
51     tax = price * 0.18
52     total = price + tax
53     return total
54 # Example usage:
55 prices = [250, 500]
56 for price in prices:
57     total_price = calculate_total(price)
58     print(f"Total Price for {price}: {total_price}")
59
```

PROBLEMS OUTPUT **TERMINAL** PORTS

✓ **TERMINAL**

Python: lab

```
Area of Rectangle with length 5 and breadth 10: 50
Perimeter of Rectangle with length 5 and breadth 10: 30
Area of Rectangle with length 7 and breadth 12: 84
```

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> ^C
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Py
on.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment_13.py
Total Price for 250: 295.0
Total Price for 500: 590.0
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:
 - o Accept marks as a parameter.
 - o Return the corresponding grade as a string.
 - o The grading logic must strictly follow the conditions below:
 - Marks ≥ 90 and $\leq 100 \rightarrow$ "Grade A"
 - Marks $\geq 80 \rightarrow$ "Grade B"
 - Marks $\geq 70 \rightarrow$ "Grade C"
 - Marks $\geq 40 \rightarrow$ "Grade D"
 - Marks $\geq 0 \rightarrow$ "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:
 - o The class
 - o The method (with parameter and return descriptions)
5. The method must be reusable and called multiple times without rewriting conditional logic.

- Given code:

```
marks = 85
```

```
if marks >= 90:
```

```
    print("Grade A")
```

```
elif marks >= 75:
```

```
    print("Grade B")
```

```
else:
print("Grade C")

marks = 72

if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
```

```
# refactor the code by defining a class named GradeCalculator.
class GradeCalculator:
    """A class to calculate grades based on marks."""
    def calculate_grade(self, marks: float) -> str:
        """Calculates the grade based on the given marks.
        Parameters:
        marks (float): The marks obtained by the student

        Returns:
        str: The grade corresponding to the marks
        """
        if marks >= 90:
            return "Grade A"
        elif marks >= 75:
            return "Grade B"
        else:
            return "Grade C"

# Example usage:
grade_calculator = GradeCalculator()
marks_list = [85, 72]
for marks in marks_list:
    grade = grade_calculator.calculate_grade(marks)
    print(f"Marks: {marks}, {grade}")
```

BLEMS	OUTPUT	TERMINAL	PORTS
TERMINAL			
Marks: 85, Grade B			
Marks: 72, Grade C			
nts : \Users\varsh\OneDrive\ドキュメント\third\AIAC>			

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

- Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

- o Identify input, processing, and output sections.
- o Convert each into a separate function.
- o Improve code readability without changing behavior.

- Sample Legacy Code:

```
num = int(input("Enter number: "))
```

```
square = num * num
```

```
print("Square:", square)
```

- Expected Output:

- o Modular code using functions like `get_input()`, `calculate_square()`, and `display_result()`.

```
90 # refactor the code to using functions like get_input(), calc
91 def get_input():
92 |     return int(input("Enter number: "))
93
94 def calculate_square(num):
95 |     return num * num
96
97 def display_result(square):
98 |     print("Square:", square)
99
100 # Example usage:
101 num = get_input()
102 square = calculate_square(num)
103 display_result(square)
```

ROBLEMS OUTPUT **TERMINAL** PORTS

✓ **TERMINAL**

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/hon313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab

IndentationError: expected an indented block after 'if' statement on

● PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/hon313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab

Enter number: 5

Square: 25

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

- o Object-Oriented principles
- o Encapsulation

Legacy Code:

```
salary = 50000
```

```
tax = salary * 0.2
```

```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.

```
110 # refactor the code using A class like EmployeeSalaryCalculator with methods and a
111 class EmployeeSalaryCalculator:
112     """A class to calculate the net salary of an employee after tax deduction."""
113     def __init__(self, salary: float):
114         """Initializes the EmployeeSalaryCalculator with the given salary.
115         Parameters:
116         salary (float): The gross salary of the employee
117         """
118         self.salary = salary
119
120     def calculate_tax(self) -> float:
121         """Calculates the tax based on the salary.
122         Returns:
123         float: The calculated tax amount
124         """
125         return self.salary * 0.2
126
127     def calculate_net_salary(self) -> float:
128         """Calculates the net salary after deducting tax.
129         Returns:
130         float: The net salary after tax deduction
131         """
132         tax = self.calculate_tax()
133         net_salary = self.salary - tax
134         return net_salary
135 # Example usage:
136 employee_salary_calculator = EmployeeSalaryCalculator(50000)
137 net_salary = employee_salary_calculator.calculate_net_salary()
138 print(f"Net Salary: {net_salary}")
```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ **TERMINAL**

```
Net Salary: 40000.0
```

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> █
```

Task 6 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.

- Focus Areas:

- o Time complexity

- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
```

```
name = input("Enter username: ")
```

```
found = False
```

```
for u in users:
```

```
    if u == name:
```

```
        found = True
```

```
print("Access Granted" if found else "Access Denied")
```

```
47
48 # refactor the code using sets or dictionaries with complexity justification Focus Area
49 def check_access(username: str, user_set: set) -> str:
50     """Checks if the given username has access based on a set of authorized users.
51     Parameters:
52     username (str): The username to check for access
53     user_set (set): A set of authorized usernames
54
55     Returns:
56     str: "Access Granted" if the username is in the set, otherwise "Access Denied"
57     """
58     return "Access Granted" if username in user_set else "Access Denied"
59 # Example usage:
60 authorized_users = {"admin", "guest", "editor", "viewer"}
61 username = input("Enter username: ")
62 access_result = check_access(username, authorized_users)
63 print(access_result)
```

PROBLEMS OUTPUT **TERMINAL** PORTS

✓ **TERMINAL**

Python: lab_assignment_13

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> ^C
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment_13.py
Enter username: Varsha
Access Denied
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic

- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Library Management System (Unstructured Version)

This code needs refactoring into a proper module with documentation.

```
library_db = {}
```

Adding first book

```
title = "Python Basics"
```

```
author = "John Doe"
```

```
isbn = "101"
```

```
if isbn not in library_db:
```

```
    library_db[isbn] = {"title": title, "author": author}
```

```
    print("Book added successfully.")
```

```
else:
```

```
    print("Book already exists.")
```

Adding second book (duplicate logic)

```
title = "AI Fundamentals"
```

```
author = "Jane Smith"
```

```
isbn = "102"
```

```
if isbn not in library_db:
```

```
    library_db[isbn] = {"title": title, "author": author}
```

```
    print("Book added successfully.")
```

```
else:
```

```
    print("Book already exists.")
```

Searching book (repeated logic structure)

```
isbn = "101"
```

```
if isbn in library_db:
```

```
    print("Book Found:", library_db[isbn])
```

```
else:
```



```
print("Book not found.")  
  
# Removing book (again repeated pattern)  
  
isbn = "101"  
  
if isbn in library_db:  
    del library_db[isbn]  
    print("Book removed successfully.")  
else:  
    print("Book not found.")  
  
# Searching again  
  
isbn = "101"  
  
if isbn in library_db:  
    print("Book Found:", library_db[isbn])  
else:  
    print("Book not found.")
```

```

1  # refactor the code by creating functions add_book(title, author, isbn), remov
2  def add_book(title: str, author: str, isbn: str) -> None:
3      """Adds a book to the library database if it does not already exist.
4      Parameters:    title (str): The title of the book
5      author (str): The author of the
6      book isbn (str): The ISBN number of the book
7      Returns:      None"""
8      if isbn not in library_db:
9          library_db[isbn] = {"title": title, "author": author}
10         print("Book added successfully.")
11     else:
12         print("Book already exists.")
13 def remove_book(isbn: str) -> None:
14     """Removes a book from the library database if it exists.
15     Parameters:    isbn (str): The ISBN number of the book to remove
16     Returns:      None"""
17     if isbn in library_db:
18         del library_db[isbn]
19         print("Book removed successfully.")
20     else:
21         print("Book not found.")
22 def search_book(isbn: str) -> None:
23     """Searches for a book in the library database by its ISBN.
24     Parameters:    isbn (str): The ISBN number of the book to search for
25     Returns:      None"""
26     if isbn in library_db:
27         print("Book Found:", library_db[isbn])
28     else:
29         print("Book not found.")
30 # Example usage:
31 library_db = {}
32 add_book("Python Basics", "John Doe", "101")
33 add_book("AI Fundamentals", "Jane Smith", "102")
34 search_book("101")
35 remove_book("101")
36 search_book("101")

```

```

hon313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/1
Book added successfully.
Book added successfully.
Book Found: {'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
Book not found.
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

```
● PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> python -m pydoc library
Book added successfully.
Book added successfully.
Book Found: {'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
Help on module library:

NAME
    library - # refactor the code by creating functions add_book(title, author, isbn), remove_book(isbn), search_book(isbn)

FUNCTIONS
    add_book(title: str, author: str, isbn: str) -> None
        Adds a book to the library database if it does not already exist.
        Parameters:    title (str): The title of the book
                       author (str): The author of the
                       book isbn (str): The ISBN number of the book
        Returns:       None

    remove_book(isbn: str) -> None
        Removes a book from the library database if it exists.
        Parameters:    isbn (str): The ISBN number of the book to remove
        Returns:       None

    search_book(isbn: str) -> None
        Searches for a book in the library database by its ISBN.
        Parameters:    isbn (str): The ISBN number of the book to search for
        Returns:       None

DATA
    library db = {'102': {'author': 'Jane Smith', 'title': 'AI Fundamental...
```

[index](#)

library [c:\users\varsh\onedrive\ドキュメント\third\aiac\library.py](#)

refactor the code by creating functions [add_book](#)(title, author, isbn), [remove_book](#)(isbn), [search_book](#)(isbn)

Functions

add_book(title: str, author: str, isbn: str) -> None

Adds a book to the library database if it does not already exist.

Parameters: title (str): The title of the book

author (str): The author of the

book isbn (str): The ISBN number of the book

Returns: None

remove_book(isbn: str) -> None

Removes a book from the library database if it exists.

Parameters: isbn (str): The ISBN number of the book to remove

Returns: None

search_book(isbn: str) -> None

Searches for a book in the library database by its ISBN.

Parameters: isbn (str): The ISBN number of the book to search for

Returns: None

Data

library_db = {'102': {'author': 'Jane Smith', 'title': 'AI Fundamentals'}}

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Bad Code Version:

```
# fibonacci bad version
```

```
n=int(input("Enter limit: "))
```

```
a=0
```

```
b=1
```

```
print(a)
```

```
print(b)
```

```
for i in range(2,n):
```

c=a+b

print(c)

a=b

b=c

```
219 def generate_fibonacci(n: int) -> list:
220     """Generates a list of Fibonacci numbers up to the nth number.
221     Parameters:
222     n (int): The number of Fibonacci numbers to generate
223     Returns:
224     list: A list containing the Fibonacci sequence up to the nth number
225     """
226     if n <= 0:
227         return []
228     elif n == 1:
229         return [0]
230     elif n == 2:
231         return [0, 1]
232     fib_sequence = [0, 1]
233     for i in range(2, n):
234         next_fib = fib_sequence[i-1] + fib_sequence[i-2]
235         fib_sequence.append(next_fib)
236     return fib_sequence
237 # Example usage and test cases:
238 n = int(input("Enter limit: "))
239 fibonacci_numbers = generate_fibonacci(n)
240 print(f"Fibonacci sequence up to {n} numbers: {fibonacci_numbers}")
```

PROBLEMS OUTPUT TERMINAL PORTS

▼ TERMINAL Python: lab_assignment_13 + ▾

Enter limit: 50
Fibonacci sequence up to 50 numbers: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, 514229, 832040, 1346269, 2178309, 3524578, 5702887, 9227465, 14930352, 24157817, 39088169, 63245986, 102334155, 165580141, 267914296, 433494437, 701408733, 1134903170, 1836311903, 2971215073, 4807526976, 7778742049]
2971215073, 4807526976, 7778742049]
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

The original code directly prints values and has poor indentation, so it may cause errors and is not reusable. The AI-refactored version uses a function with proper structure and documentation (docstring). The refactored code returns a list instead of just printing, making it reusable in other programs. It also handles edge cases (like $n \leq 0$, $n == 1$, $n == 2$) properly. Overall, the AI version is cleaner, more organized, and follows better coding practices.

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into `is_prime(n)` and `is_twin_prime(p1, p2)`.

- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version

a=11
b=13
fa=0
for i in range(2,a):
    if a%i==0:
        fa=1
fb=0
for i in range(2,b):
    if b%i==0:
        fb=1
if fa==0 and fb==0 and abs(a-b)==2:
    print("Twin Primes")
else:
    print("Not Twin Primes")
```

```
# Refactor into is_prime(n) and is_twin_prime(p1, p2). Add docstrings and optimize. Gen
def is_prime(n: int) -> bool:
    """Checks if a number is prime.
    Parameters:
    n (int): The number to check for primality
    Returns:
    bool: True if the number is prime, False otherwise"""
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

def is_twin_prime(p1: int, p2: int) -> bool:
    """Checks if two numbers are twin primes.
    Parameters:
    p1 (int): The first prime number
    p2 (int): The second prime number
    Returns:
    bool: True if the numbers are twin primes, False otherwise"""
    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2

def generate_twin_primes(start: int, end: int) -> list:
    """Generates a list of twin primes within a given range.
    Parameters:
    start (int): The starting number of the range
    end (int): The ending number of the range
    Returns:
    list: A list of tuples, each containing a pair of twin primes"""
    twin_primes = []
    for num in range(start, end - 1):
        if is_twin_prime(num, num + 2):
            twin_primes.append((num, num + 2))
    return twin_primes

start_range = int(input("Enter start of range: "))
end_range = int(input("Enter end of range: "))
twin_prime_pairs = generate_twin_primes(start_range, end_range)
print(f"Twin primes between {start_range} and {end_range}: {twin_prime_pairs}")
```

TERMINAL

```
ドキュメント/third/AIAC/lab_assignment_13.py
Enter start of range: 10
Enter end of range: 50
Twin primes between 10 and 50: [(11, 13), (17, 19), (29, 31), (41, 43)]
```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure. # Chinese Zodiac Program (Unstructured Version)

This code needs refactoring.

```
year = int(input("Enter a year: "))
```

```
if year % 12 == 0:
```

```
print("Monkey")
elif year % 12 == 1:
print("Rooster")
elif year % 12 == 2:
print("Dog")
elif year % 12 == 3:
print("Pig")
elif year % 12 == 4:
print("Rat")
elif year % 12 == 5:
print("Ox")
elif year % 12 == 6:
print("Tiger")
elif year % 12 == 7:
print("Rabbit")
elif year % 12 == 8:
print("Dragon")
elif year % 12 == 9:
print("Snake")
elif year % 12 == 10:
print("Horse")
elif year % 12 == 11:
print("Goat")
```

You must:

1. Create a reusable function: `get_zodiac(year)`
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.


```
307 # Create a reusable function: get_zodiac(year), Replace the if-elif chain with a
308 def get_zodiac(year: int) -> str:
309     """Returns the Chinese zodiac sign for a given year.
310     Parameters:
311     year (int): The year for which to determine the zodiac sign
312     Returns:
313     str: The corresponding Chinese zodiac sign"""
314     zodiac_signs = [
315         "Monkey", "Rooster", "Dog", "Pig", "Rat", "Ox",
316         "Tiger", "Rabbit", "Dragon", "Snake", "Horse", "Goat"
317     ]
318     return zodiac_signs[year % 12]
319 # Example usage:
320 year = int(input("Enter a year: "))
321 zodiac_sign = get_zodiac(year)
322 print(f"The Chinese zodiac sign for the year {year} is: {zodiac_sign}")
323
```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ **TERMINAL**

```
File "c:\Users\varsh\OneDrive\ドキュメント\third\AIAC\lab_assignment_13.py", line 278, i
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Program
/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment_13.py
Enter a year: 5
The Chinese zodiac sign for the year 5 is: Ox
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> █
```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- $18 \rightarrow 1 + 8 = 9 \rightarrow 18 \div 9 = 2 \square$ (Harshad Number)
- $19 \rightarrow 1 + 9 = 10 \rightarrow 19 \div 10 \neq \text{integer} \square$ (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

```
# Harshad Number Checker (Unstructured Version)
```

```
num = int(input("Enter a number: "))
```

```
temp = num
```

```
sum_digits = 0
```

```
while temp > 0:
```

```
    digit = temp % 10
```

```
    sum_digits = sum_digits + digit
```

```
    temp = temp // 10
```

```
if sum_digits != 0:
```

```
    if num % sum_digits == 0:
```

```
        print("True")
```

```
    else:
```

```
        print("False")
```

```
else:
```

```
    print("False")
```

You must:

1. Create a reusable function: `is_harshad(number)`
2. The function must:
 - o Accept an integer parameter.
 - o Return True if the number is divisible by the sum of its digits.
 - o Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

```
350 def is_harshad(number: int) -> bool:
351     """Checks if a number is a Harshad number.
352     A Harshad number is an integer that is divisible by the sum of its digits.
353
354     Parameters:
355     number (int): The number to check for being a Harshad number
356
357     Returns:
358     bool: True if the number is a Harshad number, False otherwise
359     """
360     if number <= 0:
361         return False # Harshad numbers are typically defined for positive integers
362
363     sum_digits = sum(int(digit) for digit in str(abs(number))) # Sum of digits
364     return sum_digits != 0 and number % sum_digits == 0 # Check divisibility
365 # Example usage:
366 num = int(input("Enter a number: "))
367 if is_harshad(num):
368     print(f"{num} is a Harshad number.")
369 else:
370     print(f"{num} is not a Harshad number.")
371
```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ **TERMINAL**  Python

```
Enter a number: 10
10 is a Harshad number.
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in $n!$ (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

Factorial Trailing Zeros (Unstructured Version)

```
n = int(input("Enter a number: "))
```

```
fact = 1
```

```
i = 1
```

```
while i <= n:
```

```
fact = fact * i
i = i + 1
count = 0
while fact % 10 == 0:
    count = count + 1
    fact = fact // 10
print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: `count_trailing_zeros(n)`
2. The function must:
 - o Accept a non-negative integer `n`.
 - o Return the number of trailing zeros in `n!`.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

```
395 def count_trailing_zeros(n: int) -> int:
396     """Counts the number of trailing zeros in the factorial of a given non-negative integer n.
397     Trailing zeros in n! are caused by the factors of 10, which are made of 2 and 5. Since the
398     Parameters:
399     n (int): The non-negative integer for which to count trailing zeros in its factorial
400     Returns:
401     int: The number of trailing zeros in n!
402     """
403     if n < 0:
404         raise ValueError("Input must be a non-negative integer.")
405     count = 0
406     power_of_5 = 5
407     while power_of_5 <= n:
408         count += n // power_of_5
409         power_of_5 *= 5
410     return count
411 # Example usage:
412 n = int(input("Enter a non-negative integer: "))
413 try:
414     trailing_zeros = count_trailing_zeros(n)
415     print(f"The number of trailing zeros in {n}! is: {trailing_zeros}")
416 except ValueError as e:
417     print(e)
418
```

PROBLEMS OUTPUT **TERMINAL** PORTS

> **TERMINAL**

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python39-64/Python.exe C:/Users/varsh/OneDrive/ドキュメント\third\AIAC/lab_assignment_13.py
Enter a non-negative integer: 10
The number of trailing zeros in 10! is: 2
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 13 (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.
- Test Cases to Design:
- Normal: $6 \rightarrow [6, 3, 10, 5, 16, 8, 4, 2, 1]$
- Edge: $1 \rightarrow [1]$
- Negative: -5
- Large: 27 (well-known long sequence)
- Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

- Takes an integer n as input.
- Generates the Collatz sequence (also called the $3n+1$ sequence).
- The rules are:
 - o If n is even $\rightarrow$ next = $n / 2$.
 - o If n is odd $\rightarrow$ next = $3n + 1$.

- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even $\rightarrow 6/2 = 3$)
- 3 (odd $\rightarrow 3*3+1 = 10$)
- 10 (even $\rightarrow 10/2 = 5$)
- 5 (odd $\rightarrow 3*5+1 = 16$)
- 16 (even $\rightarrow 16/2 = 8$)
- 8 (even $\rightarrow 8/2 = 4$)
- 4 (even $\rightarrow 4/2 = 2$)
- 2 (even $\rightarrow 2/2 = 1$)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

```

3 def test_generate_collatz_sequence():
4     assert generate_collatz_sequence(6) == [6, 3, 10, 5, 16, 8, 4, 2, 1], "Test case 1 failed"
5     assert generate_collatz_sequence(27) == [27, 82, 41, 124, 62, 31, 94, 47, 142, 71, 214, 107, 322, 161, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1], "Test case 2 failed"
6     assert generate_collatz_sequence(1) == [1], "Test case 3 failed"
7     try:
8         generate_collatz_sequence(-5)
9         assert False, "Test case 4 failed: ValueError not raised for negative input"
10    except ValueError:
11        pass
12    try:
13        generate_collatz_sequence(0)
14        assert False, "Test case 5 failed: ValueError not raised for zero input"
15    except ValueError:
16        pass
17    print("All test cases passed!")

```

PROBLEMS 1 OUTPUT **TERMINAL** PORTS

✓ **TERMINAL**

```

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> python c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/collatz.py
All test cases passed!
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

Task 14 (Lucas Number Sequence – Test Case Design)

- Function: Generate Lucas sequence up to n terms.

(Starts with 2,1, then $F_n = F_{n-1} + F_{n-2}$)

- Test Cases to Design:

- Normal: $5 \rightarrow [2, 1, 3, 4, 7]$
- Edge: $1 \rightarrow [2]$
- Negative: $-5 \rightarrow \text{Error}$
- Large: 10 (last element = 76).
- Requirement: Validate correctness with pytest.

```

174 def generate_lucas_sequence(n: int) -> list:
175     """Generates the Lucas sequence up to n terms.
176     The Lucas sequence starts with 2, 1, and each subsequent term is the sum of the two preceding terms
177     Parameters:
178     n (int): The number of terms to generate in the Lucas sequence
179     Returns:
180     list: A list containing the first n terms of the Lucas sequence
181     """
182     if n <= 0:
183         | raise ValueError("Input must be a positive integer.")
184     if n == 1:
185         | return [2]
186     lucas_sequence = [2, 1]
187     for i in range(2, n):
188         | lucas_sequence.append(lucas_sequence[i-1] + lucas_sequence[i-2])
189     return lucas_sequence
190 def test_generate_lucas_sequence():
191     assert generate_lucas_sequence(5) == [2, 1, 3, 4, 7], "Test case 1 failed"
192     assert generate_lucas_sequence(1) == [2], "Test case 2 failed"
193     try:
194         | generate_lucas_sequence(-5)
195         | assert False, "Test case 3 failed: ValueError not raised for negative input"
196     except ValueError:
197         | pass
198     assert generate_lucas_sequence(10) == [2, 1, 3, 4, 7, 11, 18, 29, 47, 76], "Test case 4 failed"

```

Task 15 (Vowel & Consonant Counter – Test Case Design)

- Function: Count vowels and consonants in string.
- Test Cases to Design:
- Normal: "hello" $\rightarrow (2,3)$
- Edge: "" $\rightarrow (0,0)$
- Only vowels: "aeiou" $\rightarrow (5,0)$
- Large: Long text
- Requirement: Validate correctness with pytest.

```

1  def count_vowels_consonants(s: str) -> tuple:
2      """Counts the number of vowels and consonants in a given string.
3      Parameters:
4      s (str): The input string to analyze
5      Returns:
6      tuple: A tuple containing the count of vowels and consonants in the format (vowels, consonants)"""
7      vowels = 'aeiouAEIOU'
8      vowel_count = sum(1 for char in s if char in vowels)
9      consonant_count = sum(1 for char in s if char.isalpha() and char not in vowels)
10     return (vowel_count, consonant_count)
11
12  def test_count_vowels_consonants():
13      assert count_vowels_consonants("hello") == (2, 3), "Test case 1 failed"
14      assert count_vowels_consonants("") == (0, 0), "Test case 2 failed"
15      assert count_vowels_consonants("aeiou") == (5, 0), "Test case 3 failed"
16      assert count_vowels_consonants("This is a long text with multiple vowels and consonants.") == (16, 29)
17      print("All test cases passed!")

```

PROBLEMS 1 OUTPUT **TERMINAL** PORTS

TERMINAL

```

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.
/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment_13.py
All test cases passed!
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> 

```